DeployGuard — FYP-II Final Presentation Data

This document is the authoritative content source for preparing the final presentation. It reflects the implemented product and preserved certification evidence. Live-cloud claims are deliberately separated from structural verification.

SECTION 1 — PROJECT OVERVIEW AND SHARED FOUNDATION

Slide 1 — DeployGuard: Deterministic Cloud Deployment Control Plane

Presenter: Hassan Sajjad

Purpose: Open the defense with the product identity, team, and central promise.

Presentation Content:

- DeployGuard is a platform-as-a-service-style control plane for deploying repository applications to AWS.
- It converts a selected source revision and explicit configuration into a traceable cloud release.

- The core promise is deterministic delivery: build, provision, verify, promote, observe, recover, and destroy through one governed lifecycle.
- Team members: Hassan Sajjad, Tania Khawar, and Faria Fatima.
- FYP-II focus: implementation maturity, end-to-end integration, lifecycle reliability, observability, and defensible verification.

Visual Recommendation: A clean title slide with the DeployGuard logo and a single flow: Repository → DeployGuard → AWS Runtime.

Defense Notes: Describe DeployGuard as a control plane, not merely a deployment script or dashboard. The differentiator is that it preserves source, image, configuration, infrastructure, and release identity throughout the lifecycle.

Slide 2 — Problem Statement

Presenter: Hassan Sajjad

Purpose: Establish why the project is needed.

Presentation Content:

- Cloud deployment forces small teams to combine source control, container builds, infrastructure provisioning, runtime diagnosis, security scanning, cost review, and cleanup.
- These tools expose different identifiers and failure formats, so developers struggle to answer: what was deployed, why did it fail, and what is safe to do next?
- Common risks include configuration drift, mutable artifacts, incomplete cleanup, excessive cloud permissions, hidden cost, and rollback based on guesses.
- Existing automation often reports a failed command without identifying whether responsibility belongs to the application, platform, or cloud provider.
- DeployGuard unifies those concerns without hiding the underlying evidence.

Visual Recommendation: A “before DeployGuard” fragmentation diagram with GitHub, container tooling, Terraform, AWS, logs, security, and billing around a confused developer.

Defense Notes: Keep the problem technical and operational. The project is not claiming that individual cloud tools are inadequate; the problem is coordinating them into a consistent, auditable lifecycle.

Slide 3 — Objectives and Final Product Scope

Presenter: Hassan Sajjad

Purpose: State what was built and how success is defined.

Presentation Content:

- Provide self-service onboarding from a GitHub repository and selected source revision.
- Detect a reproducible application build target and runtime port, then produce immutable container images.
- Provision AWS infrastructure through Terraform and operate application releases through Amazon ECS.
- Enforce admission checks for permissions, configuration, security, cost, and managed-database readiness.
- Support first deployment, release-only redeployment, exact historical rollback, and verified destruction.
- Present live status, logs, metrics, cost context, audit history, and evidence-grounded AI assistance.
- Package the complete product for repeatable local operation with container orchestration.

Visual Recommendation: Seven objective cards grouped under Build, Cloud, Lifecycle, Governance, Observability, Intelligence, and Portability.

Defense Notes: Explicitly separate product features from proof status. An implemented path may be structurally verified while a particular cloud engine or failure scenario still requires fresh live-cloud evidence.

Slide 4 — Target Users and End-to-End User Journey

Presenter: Hassan Sajjad

Purpose: Explain the product through the developer's experience.

Presentation Content:

- A developer connects a GitHub App installation and selects a repository, source branch, and one or more service directories.
- DeployGuard analyzes each service, resolves its build and port contract, and collects service-scoped environment configuration.
- The developer may attach a supported managed database and reviews deployment readiness before starting.
- The platform builds, scans, publishes, provisions, starts, verifies, and promotes the release while exposing progress.
- After deployment, the developer sees the public endpoint, infrastructure, pipeline history, live logs, metrics, and troubleshooting evidence.
- The same project can be redeployed, rolled back to an immutable stable release, or destroyed with explicit verification.

Visual Recommendation: A numbered user-journey ribbon from Connect Repository to Operate or Destroy.

Defense Notes: Mention that multi-service selection is supported and configuration belongs to the selected service. The journey is designed to reduce ambiguity, not to remove control from the developer.

Slide 5 — Complete System Architecture

Presenter: Hassan Sajjad

Purpose: Give the panel a shared mental model before component detail.

Presentation Content:

- Presentation layer: React web application for projects, pipeline, infrastructure, monitoring, troubleshooting, billing, and administration.
- Control plane: NestJS APIs and workers for authentication, repository analysis, deployment admission, orchestration, lifecycle actions, audit, and billing.
- Durable control data: PostgreSQL for projects, configurations, generations, stable releases, evidence, infrastructure state, and audit records.
- Asynchronous execution: Redis-backed BullMQ queues for deployment, lifecycle, and emergency-cleanup work.
- Execution plane: GitHub Actions uses short-lived AWS federation to build, scan, publish, and invoke Terraform and ECS operations.
- Cloud plane: ECR, ECS on Fargate, Application Load Balancers, CloudWatch, Secrets Manager, EFS, service discovery, and managed database containers.
- Observability plane: CloudWatch evidence projected through APIs, Prometheus metrics, and Grafana dashboards.

Visual Recommendation: Layered architecture with arrows from browser to control plane, database and queues, workflow execution, then AWS services.

Defense Notes: Clarify that the control plane owns deployment decisions while the workflow is the privileged execution mechanism. PostgreSQL stores the durable product record; Redis coordinates asynchronous work rather than replacing that record.

Slide 6 — Technology Stack and Why It Fits

Presenter: Hassan Sajjad

Purpose: Connect technology choices to product requirements.

Presentation Content:

- Frontend: React, React Router, Vite, and charting components for a responsive single-page application.
- Backend: NestJS and TypeScript for modular APIs, validation, dependency injection, and background orchestration.
- Data and jobs: PostgreSQL with TypeORM, plus Redis and BullMQ for retryable asynchronous tasks.
- Delivery: GitHub App integration, GitHub Actions, Docker-compatible images, immutable ECR repositories, and Trivy scanning.
- Infrastructure: Terraform with encrypted, versioned S3 remote state and native lockfiles.
- Runtime: Amazon ECS Fargate behind per-service Application Load Balancers.
- Operations: CloudWatch, Prometheus, Grafana, structured audit data, and evidence-based AI analysis.
- Commerce: Stripe test-mode subscription and usage integration.

Visual Recommendation: A technology map grouped by Frontend, Control Plane, Delivery, AWS Runtime, Observability, and Billing.

Defense Notes: If asked why this is not a single monolith, explain that durable state, job execution, privileged workflow actions, and runtime observation have different reliability and security boundaries.

Slide 7 — Canonical Identity and Deployment Lifecycle

Presenter: Hassan Sajjad

Purpose: Explain the deterministic data model that connects every stage.

Presentation Content:

- Build Target records how a specific service is built and started.
- Runtime Config Revision records the environment and non-secret runtime contract without exposing secret values.
- Deployment Generation records one concrete deployment attempt

and its source, artifacts, stages, and evidence.
- Stable Release points only to a generation that passed required verification.
- Failure Diagnostic stores deterministic code, stage, owner, affected service, root cause, and retry guidance.
- Primary sequence: resolve exact source → build and validate → publish immutable image → ensure infrastructure → satisfy database prerequisite → stabilize runtime → verify endpoint → promote.
- These linked identities prevent a later action from silently changing the meaning of an earlier release.

Visual Recommendation: An entity chain with identifiers flowing across Source, Build Target, Config Revision, Generation, Image Digest, Task Revision, and Stable Release.

Defense Notes: This is the conceptual center of the project. “Latest” is not accepted as a release identity; the system retains exact evidence so redeploy and rollback decisions are explainable.

Slide 8 — Team Responsibilities and Presentation Route

Presenter: Hassan Sajjad

Purpose: Make ownership explicit and transition into individual contributions.

Presentation Content:

- Hassan Sajjad: project leadership, deployment architecture, infrastructure as code, AWS lifecycle, security and cost gates, release reliability, and integration certification.
- Tania Khawar: frontend experience, onboarding and deployment visualization, real-time monitoring presentation, and AI-assisted troubleshooting experience.
- Faria Fatima: backend orchestration, authentication and repository integration, persistence, queues, auditability, billing integration,

and containerized product operation.
- Shared work: requirements validation, cross-layer integration, test support, documentation, and final demonstration.
- Presentation order follows the implemented architecture: shared foundation → cloud and DevOps → frontend and AI → backend and product integration.

Visual Recommendation: Three-column responsibility matrix with a shared-integration band across the bottom.

Defense Notes: Ownership describes primary responsibility, not isolated work. Emphasize that all three layers meet through explicit contracts and were integrated as one product.

SECTION 2 — HASSAN SAJJAD: CLOUD, DEVOPS, SECURITY, COST, AND RELIABILITY

Slide 9 — Control-Plane, Terraform, and ECS Ownership

Presenter: Hassan Sajjad

Purpose: Defend the architectural boundary that prevents competing authorities.

Presentation Content:

- Terraform owns infrastructure existence, networking, load balancers, IAM, EFS, service discovery, database infrastructure, state, and final destruction.
- DeployGuard owns admission, database readiness decisions,

runtime-success classification, verification, stable-release promotion, and manual rollback policy.
- ECS owns low-level task placement, scaling, rolling deployment, and stability primitives.
- Application services protect externally managed desired count and active task revision from later topology applies.
- A database-attached application is created at zero desired tasks, then raised by DeployGuard only after its database prerequisite succeeds.
- Unrelated services do not wait on another service's database.

Visual Recommendation: Three overlapping authority circles with non-overlapping responsibility lists and a shared lifecycle arrow.

Defense Notes: This boundary avoids configuration fights. Terraform can still destroy the service, but it does not undo a legitimate post-create scale-up or release-only task revision selected by DeployGuard.

Slide 10 — AWS Runtime Topology and Network Isolation

Presenter: Hassan Sajjad

Purpose: Show the concrete cloud architecture of a deployed project.

Presentation Content:

- Each project receives an ECS cluster and each application service receives its own task definition, service, log group, target group, listener, and load balancer path.
- Application tasks run on Fargate and are reached through an Application Load Balancer health-checked target group.
- The application security group accepts only the declared service and probe ports from the load-balancer security group.
- A managed database is discovered through private service discovery and accepts its engine port only from its attached application security group.
- Encrypted EFS provides persistent database storage with transit

encryption and a restricted access point.
- CloudWatch captures task logs and operational metrics; Secrets Manager supplies platform-owned database credentials.

Visual Recommendation: AWS topology diagram: Internet → ALB → app task; app task → database task → EFS; both → CloudWatch; secrets → tasks.

Defense Notes: Use “network-isolated with private service discovery,” not “private-subnet database.” Security-group relationships are the implemented inbound boundary.

Slide 11 — Infrastructure as Code and State Safety

Presenter: Hassan Sajjad

Purpose: Explain repeatability, concurrency protection, and recoverability.

Presentation Content:

- Terraform modules materialize project-specific runtime inputs into deterministic infrastructure plans.
- Remote state uses an encrypted, versioned S3 bucket with public access blocked and native lockfile coordination.
- State admission validates backend ownership and safety before cloud mutation begins.
- Planning and applying are separate stages; planning failure is recorded with its own deterministic code and stage.
- Safety snapshots, lock and heartbeat records, workflow evidence, and cloud observation support recovery from interrupted operations.
- Lifecycle rules permit ECS-managed desired count and active release revision while retaining Terraform’s authority to delete resources.

Visual Recommendation: State-safety sequence: Validate Backend →

Acquire Lock → Plan → Apply → Persist Evidence → Release Lock.

Defense Notes: If older documentation mentions a separate lock database, clarify that the implemented system uses S3's native lockfile mechanism. The source implementation is authoritative for the final architecture.

Slide 12 — Immutable Build, Scan, and Delivery Pipeline

Presenter: Hassan Sajjad

Purpose: Demonstrate how source becomes a verifiable runtime artifact.

Presentation Content:

- The workflow checks out the exact selected source revision rather than an ambiguous moving reference.
- Repository analysis resolves service root, build contract, start command, health route, and one canonical runtime port.
- Images are built and locally validated before publication to immutable ECR repositories.
- Trivy produces vulnerability evidence and can enforce configured severity policy before deployment proceeds.
- The workflow validates immutable inputs, performs Terraform init, validate, plan, and apply for infrastructure, then verifies ECS and the public endpoint.
- Final evidence binds source identity, image digest, runtime configuration, infrastructure action, and release result.

Visual Recommendation: Horizontal delivery pipeline with red stop-gates at Input Validation, Local Runtime, Security Scan, Infrastructure, and Public Verification.

Defense Notes: Build-system internals are deliberately hidden behind a canonical build contract. What matters at the control-plane boundary is reproducibility, local validation, immutable publication, and recorded

evidence.

Slide 13 — Security Architecture and Least Privilege

Presenter: Hassan Sajjad

Purpose: Show that security is enforced throughout the lifecycle.

Presentation Content:

- GitHub Actions obtains short-lived AWS credentials through OpenID Connect; long-lived cloud access keys are not the normal execution model.
- Capability admission verifies that the configured role can perform the exact AWS operations required by the selected action.
- Third-party workflow actions are pinned immutably, and container images are referenced by digest after publication.
- Secrets are encrypted at rest in the control plane, injected only into the correct service, and redacted from logs and AI evidence.
- HTTP protections include Helmet, restricted CORS, validated request DTOs, signed secure session cookies, OAuth state validation, and role separation.
- Audit records capture actor, action, resource, result, network context, and sanitized metadata.

Visual Recommendation: Defense-in-depth shield with Identity, Input, Artifact, Secret, Runtime, and Audit layers.

Defense Notes: Trivy is one control, not the whole security story. Stress temporary federation, explicit capability checks, service-scoped secrets, immutable references, and auditability.

Slide 14 — FinOps and Billing-Aware Admission

Presenter: Hassan Sajjad

Purpose: Present cost as a deployment decision rather than a later surprise.

Presentation Content:

- DeployGuard computes a structured infrastructure estimate with service-level and resource-level breakdowns.
- Cost evidence is associated with the deployment context so a reviewer can see what is expected to create spend.
- Configurable thresholds and approval states can prevent a deployment from proceeding without acknowledgement.
- Billing entitlements and usage are evaluated by the control plane rather than trusted from the browser.
- Stripe integration supports test-mode plans, checkout, customer state, invoices, and signed webhook processing when configured.
- Cloud cost estimates and product subscription billing are separate concerns: one predicts AWS usage; the other governs DeployGuard access.

Visual Recommendation: Split panel: Estimated AWS Resources and DeployGuard Plan/Entitlement, joined at the admission gate.

Defense Notes: Do not claim exact future AWS bills. Estimates are decision support. Also state clearly that the preserved billing demonstration is test-mode integration, not evidence of live commercial charging.

Slide 15 — Managed Databases and Service-Specific Readiness

Presenter: Hassan Sajjad

Purpose: Explain database persistence without creating a global deployment bottleneck.

Presentation Content:

- Supported engine contracts cover PostgreSQL, MySQL, and MongoDB with engine-specific health behavior.
- Credentials are generated and stored through Secrets Manager; data is persisted on encrypted EFS.
- Database state is reconciled before deployment into HEALTHY, RECOVERABLE, DATA_LOST_RESET_REQUIRED, STALE_METADATA, or IDENTITY_MIGRATION_REQUIRED.
- Deployment proceeds only when the authoritative reconciliation result explicitly allows it.
- PostgreSQL and MongoDB require container readiness; MySQL additionally requires successful application-user grant reconciliation.
- Only the attached application waits for its database; independent services continue through their own lifecycle.
- A fresh deployment never silently deletes healthy persistent data.

Visual Recommendation: Three engine cards feeding a per-service readiness gate, with unrelated services bypassing that gate.

Defense Notes: Distinguish structural support from live certification. The current source contains the engine-specific contracts, while some engine and dynamic-network cases remain listed for fresh AWS proof.

Slide 16 — Release Reliability, Recovery, and Verified Cleanup

Presenter: Hassan Sajjad

Purpose: Close the cloud section with lifecycle guarantees and honest evidence.

Presentation Content:

- First deployment provisions topology, starts prerequisites, stabilizes ECS, verifies the public route, and only then promotes a stable release.
- Release-only redeployment builds a new immutable artifact and updates ECS directly without a full infrastructure apply.

- Manual rollback selects an exact previous stable task revision; it does not rebuild source or guess historical configuration.
- Reset and Deploy Fresh performs reconciliation first, creates a new generation, and preserves healthy persistent data.
- Destroy requires explicit admission, executes Terraform destruction, inventories cloud resources, verifies deletion, and finalizes control-plane state.
- Failures are attributed to platform, repository application, external provider, or unverified, with stage-specific evidence.
- Preserved certification demonstrates clean deploys and a deploy–redeploy–rollback–destroy lifecycle; additional database isolation, topology-reapply, and repeat-destroy scenarios remain candidates for fresh cloud proof.

Visual Recommendation: Lifecycle loop with Deploy, Verify, Promote, Redeploy, Rollback, and Destroy; add green “preserved evidence” and amber “fresh proof pending” labels.

Defense Notes: Never present structural tests as live-cloud proof. The strongest defense is traceability: every transition has an immutable identity, terminal state, and evidence record.

SECTION 3 — TANIA KHAWAR: FRONTEND, MONITORING EXPERIENCE, AND AI ASSISTANCE

Slide 17 — Frontend Information Architecture

Presenter: Tania Khawar

Purpose: Show how the interface turns a complex platform into understandable tasks.

Presentation Content:

- The React single-page application separates public entry, authentication, developer workspaces, and administration.
- Developer routes include Dashboard, Projects, New Project, Project Overview, Pipeline, Infrastructure, Monitoring, Settings, Troubleshooting, and Billing.
- Route protection prevents unauthenticated access, while role checks separate developer and administrator experiences.
- Project pages share a stable navigation context so users do not lose the selected project while moving between lifecycle views.
- Polling and server-sent events keep long-running cloud operations visible without requiring manual refresh.
- UI copy translates provider concepts into product states while retaining detailed evidence for technical users.

Visual Recommendation: Annotated sitemap with public, developer, project-detail, billing, and admin route groups.

Defense Notes: Explain the distinction between simplifying and hiding. Summary cards support quick decisions, while infrastructure, pipeline, logs, and diagnostics preserve technical detail.

Slide 18 — Repository Onboarding and Deployment Configuration UX

Presenter: Tania Khawar

Purpose: Demonstrate the guided path from GitHub connection to deployment readiness.

Presentation Content:

- The New Project flow first verifies the user's GitHub App connection and available installation repositories.

- Users select a repository and source branch, then choose one or more service directories for deployment.
- Each service receives its own environment configuration, detected port contract, and optional managed-database attachment.
- The interface supports bulk environment-variable paste while keeping service ownership visible.
- Readiness feedback blocks incomplete or contradictory configurations before a workflow is dispatched.
- The final review summarizes repository, services, runtime settings, database choices, and expected action.

Visual Recommendation: A five-step wizard storyboard using actual screenshots: Connect, Select, Configure Services, Attach Database, Review.

Defense Notes: For the presentation screenshot, crop any internal builder implementation label. Discuss the user-facing contract as automated container build and validation.

Slide 19 — Deployment Pipeline and Current-State Visualization

Presenter: Tania Khawar

Purpose: Explain how the UI communicates asynchronous deployment truth.

Presentation Content:

- The dashboard highlights active deployments, progress, projects requiring attention, and recently used projects.
- Project Overview reconciles stored project information with active operation and cloud-derived current state.
- Pipeline presents deployment generations, stage progression, terminal status, failure evidence, and recovery actions.
- Long-running actions are represented as operations with explicit states rather than optimistic browser-only changes.
- Stable release and current generation are shown separately so

the user can distinguish “attempted” from “verified and promoted.”
- Retry, reset, redeploy, rollback, and destroy controls are exposed only when their lifecycle conditions permit them.

Visual Recommendation: One real Pipeline screenshot annotated with Generation, Stage, Stable Release, Evidence, and Allowed Actions.

Defense Notes: The interface does not decide success. It renders authoritative backend and cloud evidence. This separation prevents a refreshed browser or interrupted connection from changing deployment truth.

Slide 20 — Monitoring, Metrics, and Live Logs

Presenter: Tania Khawar

Purpose: Show the implemented observability experience, including Prometheus and Grafana.

Presentation Content:

- Monitoring presents CPU, memory, load-balancer latency, and healthy versus unhealthy target signals derived from AWS observations.
- Live CloudWatch logs are streamed to the browser through server-sent events with reconnection and filtering behavior.
- Project infrastructure views connect service topology, public endpoint, region, runtime state, and supporting services.
- The backend exposes authenticated Prometheus metrics for control-plane and pipeline health.
- Grafana is provisioned with data sources and dashboards for an operator-level view of platform behavior.
- Developer monitoring and operator monitoring are complementary: project users see scoped evidence, while operators see system-wide service health.

Visual Recommendation: Three-panel composition: Monitoring charts,

live log stream, and a Grafana dashboard screenshot.

Defense Notes: Prometheus is not presented as a replacement for CloudWatch. CloudWatch is the AWS runtime evidence source; Prometheus and Grafana provide control-plane aggregation and operational dashboards.

Slide 21 — Evidence-Grounded AI Troubleshooting

Presenter: Tania Khawar

Purpose: Defend the AI feature as bounded assistance rather than deployment authority.

Presentation Content:

- The troubleshooting flow collects persisted pipeline events, sanitized logs, structured diagnostics, and relevant runtime evidence.
- Evidence is bounded, deduplicated, and filtered before it reaches the model provider.
- Sensitive patterns such as credentials, tokens, authorization headers, private keys, and secret URLs are redacted.
- Model output must follow a structured schema and cite only evidence identifiers supplied in the request.
- Authoritative failure owner, root cause, and retry policy cannot be overwritten by generated advice.
- If the provider is unavailable or its response fails validation, DeployGuard returns a deterministic evidence-only fallback.
- AI recommendations are advisory; lifecycle decisions remain deterministic.

Visual Recommendation: Guardrailed AI flow: Sanitized Evidence → Bounded Model Request → Schema Validation → Advice, with Deterministic Fallback below.

Defense Notes: The safe answer to “Can AI trigger deployment?” is no.

It explains evidence and possible remediation; it does not promote a release, change ownership, or bypass admission.

Slide 22 — Frontend Validation and Demonstration Evidence

Presenter: Tania Khawar

Purpose: Show how the user experience was tested and what the demo will prove.

Presentation Content:

- Frontend verification covers route protection, project visibility, lifecycle actions, state projection, infrastructure, monitoring, billing, and administration.
- An isolated full-stack run exercised browser mutations and cross-page surfaces against a real local API and database.
- That run reached a READY current-state result and verified that data remained consistent across core screens.
- The isolated GitHub repository-selection step remained an expected blocker because the test user had no GitHub App installation; it was not misreported as a product pass.
- The final demo should show actual page transitions and stored evidence rather than only static mockups.
- A prepared screenshot fallback protects the defense from network or cloud-session instability.

Visual Recommendation: Test-evidence collage with green checks for authenticated surfaces and one amber, clearly labeled external-integration prerequisite.

Defense Notes: Emphasize honest status semantics: PASS means executed proof, EXPECTED BLOCKER means a known missing external prerequisite, and UNVERIFIED means no sufficient executable evidence.

SECTION 4 — FARIA FATIMA: BACKEND, DATA, CONTAINERIZATION, AND PRODUCT INTEGRATION

Slide 23 — Backend Control-Plane Architecture

Presenter: Faria Fatima

Purpose: Explain how backend modules coordinate the product.

Presentation Content:

- NestJS organizes authentication, users, projects, deployment orchestration, observability, AI, administration, audit, export, and billing into explicit modules.
- Controllers validate transport requests and delegate lifecycle decisions to services rather than embedding deployment logic in routes.
- PostgreSQL stores durable product truth through TypeORM entities and migrations.
- Redis-backed BullMQ queues isolate long-running deployment, lifecycle, and emergency-cleanup jobs from request-response handling.
- Workers apply retry, backoff, concurrency, and completed-job retention policies while persisting durable results in PostgreSQL.
- Global validation rejects unknown fields, and centralized HTTP protections apply consistently across modules.

Visual Recommendation: Backend component diagram: Controllers → Domain Services → PostgreSQL, with Queues/Workers beside the

synchronous path and adapters to external providers.

Defense Notes: Redis queue state is operational coordination, not the only record of a deployment. The durable generation, events, evidence, and terminal result are stored in PostgreSQL.

Slide 24 — Authentication, GitHub Integration, and Source Ingestion

Presenter: Faria Fatima

Purpose: Demonstrate controlled access to users and repository source.

Presentation Content:

- Users can authenticate through local credentials or GitHub OAuth; signed HTTP-only cookies carry separate developer and administrator sessions.
- Local passwords use salted key derivation, and OAuth state is validated to prevent request forgery.
- GitHub App installation tokens are generated for short-lived repository access rather than storing a broad permanent personal token.
- Repository selection is restricted to installations available to the authenticated user.
- Source ingestion performs a shallow checkout for the selected reference, resolves it to an exact revision, and applies path-containment checks.
- Repository analysis creates service-specific build targets and rejects ambiguous or unsupported runtime contracts before cloud work begins.

Visual Recommendation: Trust flow: User Session → GitHub App Installation → Short-Lived Token → Exact Source Checkout → Analyzed Service Contract.

Defense Notes: Differentiate user OAuth from GitHub App repository

access. OAuth identifies and links the user; the App installation token authorizes repository operations for a limited time and scope.

Slide 25 — Data Model, Configuration, and Secret Handling

Presenter: Faria Fatima

Purpose: Explain how the backend preserves traceability and confidentiality.

Presentation Content:

- Core records include projects, deployable services, build-target revisions, runtime-config revisions, deployment generations, stable releases, and pipeline events.
- Additional records cover managed databases, environment routes, FinOps decisions, infrastructure evidence, state operations, storage, observability, AI analysis, notifications, and billing.
- Environment variables are service-scoped so values cannot silently leak between applications in a multi-service project.
- Sensitive values are encrypted with authenticated encryption before storage and are decrypted only for authorized execution paths.
- Runtime configuration revisions preserve deploy-time identity without displaying plaintext secrets in evidence or UI responses.
- Database migrations make schema evolution repeatable in local and containerized environments.

Visual Recommendation: Simplified entity relationship diagram centered on Project → Service → Config Revision → Generation → Stable Release.

Defense Notes: Avoid saying that an encrypted secret is safe merely because it is hidden in the interface. The defense includes encryption at rest, scoped access, controlled injection, redaction, and audit trails.

Slide 26 — Orchestration, Failure Evidence, Audit, and Notifications

Presenter: Faria Fatima

Purpose: Show how backend operations remain understandable during success and failure.

Presentation Content:

- Admission validates source identity, service contracts, managed-database state, AWS capabilities, runtime secrets, billing entitlement, and action eligibility.
- A lifecycle action creates or references a generation, dispatches asynchronous work, consumes workflow evidence, and persists terminal state.
- Failure diagnostics carry deterministic code, lifecycle stage, owner, affected service, root cause, evidence, and retry guidance.
- Audit logs preserve who performed an action, what resource was affected, whether it succeeded, and sanitized request context.
- Notification adapters can publish operational email subscriptions through AWS SNS when live configuration and credentials are available.
- Recovery actions reuse persisted evidence and reconciliation rules instead of treating every failure as a clean restart.

Visual Recommendation: Sequence diagram: API Admission → Queue → Workflow → Evidence Ingestion → State Transition → Audit/Notification → UI.

Defense Notes: A failure code alone is insufficient. DeployGuard combines code, stage, owner, service scope, and evidence so the next action can be safe and specific.

Slide 27 — Billing, Entitlements, and Administrative Control

Presenter: Faria Fatima

Purpose: Explain product governance beyond deployment mechanics.

Presentation Content:

- Billing services manage plan catalog, trial state, customer linkage, checkout sessions, subscription status, usage, and invoices.
- Signed webhook validation is required before external billing events alter durable subscription state.
- Entitlement checks are performed server-side for protected capabilities.
- The billing UI clearly indicates test mode in the demonstrated configuration.
- Administrator tools expose service readiness, users and roles, canonical project states, and sanitized audit events.
- Separate admin authentication and role enforcement reduce the chance that ordinary developer access becomes platform-wide control.

Visual Recommendation: Two-column view: Developer Billing Experience and Administrator Governance, both backed by server-side policy.

Defense Notes: Present billing as integrated product capability, not as proof of a production financial operation. The defensible result is the implemented test-mode path, signed event handling, and server-side entitlement model.

Slide 28 — Portable Containerized Product Setup

Presenter: Faria Fatima

Purpose: Show that the full platform can run consistently outside an individual development machine.

Presentation Content:

- The local product stack is defined as containers for PostgreSQL, one-shot database migration, backend, frontend, Prometheus, and Grafana.
- Health checks and dependency ordering prevent the application from racing an unavailable database or unfinished migration.
- Named volumes preserve database and monitoring data across ordinary restarts.
- The backend image uses a multi-stage build, includes the required infrastructure tooling, and runs as a non-root user.
- The frontend is compiled once and served by an unprivileged Nginx runtime on its declared port.
- Environment-specific secrets remain external configuration; they are not baked into images or committed defaults.

Visual Recommendation: Container composition diagram with health arrows and persistent-volume icons.

Defense Notes: For a teammate handoff, the required inputs are Docker, the supplied environment configuration, and external credentials appropriate to the features being demonstrated. Local platform login credentials are distinct from Prometheus authentication and Grafana operator credentials.

Slide 29 — Verification Strategy and Honest Limitations

Presenter: Faria Fatima

Purpose: Explain what was tested and prevent overclaiming.

Presentation Content:

- Verification spans backend compilation and contracts, frontend build and behavior checks, Terraform validation and tests, workflow contracts, container health, and full-stack browser flows.
- Stored certification evidence records two clean deployments and one complete deploy, release-only redeploy, rollback, and destroy sequence.

- That evidence includes a successful public health response and verified removal of project-scoped resources while shared platform components remained intact.
- Structural checks verify architecture and contracts; only execution against AWS can establish live end-to-end behavior for a specific revision and configuration.
- Fresh live proof is still appropriate for MySQL dynamic task-network grants, MongoDB, database-failure isolation and recovery, topology reapply preserving the active release, and repeat-destroy idempotency.
- External integration prerequisites are labeled instead of being converted into false passes.

Visual Recommendation: Evidence matrix with rows for Local, Contract, Full Stack, and Live Cloud, and columns for Deploy, Observe, Redeploy, Rollback, Destroy, and Database Cases.

Defense Notes: If challenged on an unverified case, state the exact boundary and the test required. Honest evidence classification is itself a reliability feature.

Slide 30 — Integrated Result and Conclusion

Presenter: Faria Fatima

Purpose: Close the presentation with the system-level achievement and future direction.

Presentation Content:

- DeployGuard integrates repository onboarding, deterministic build contracts, infrastructure provisioning, ECS runtime control, security, cost review, observability, AI assistance, billing, and lifecycle recovery.
- Its central contribution is an explicit authority model backed by immutable identities and persisted evidence.
- Developers receive one coherent workflow without losing access to source, artifact, infrastructure, runtime, and failure details.

- The team produced an operational control plane, a complete web experience, a portable container setup, and a repeatable verification framework.
- Immediate next work is focused certification of the remaining live-cloud database and idempotency scenarios, not an architectural rewrite.
- Final message: DeployGuard makes deployment state explainable, reversible, and governable.

Visual Recommendation: Return to the opening Repository → DeployGuard → AWS diagram, now surrounded by Security, Cost, Observability, AI, Audit, and Recovery outcomes.

Defense Notes: End on the system property, not a feature count. The project's value is that every operation has a clear owner, immutable identity, observable outcome, and safe next step.

A. Presenter Distribution

Slide Range	Presenter	Topics
1–8	Hassan Sajjad	Problem, objectives, architecture, stack, lifecycle model, and team ownership
9–16	Hassan Sajjad	AWS, Terraform, delivery, security, FinOps, databases, releases, recovery, and cleanup
17–22	Tania Khawar	Frontend experience, onboarding, pipeline visualization, monitoring, Grafana, Prometheus, and AI assistance
23–30	Faria Fatima	Backend, authentication, GitHub integration, persistence, queues, audit, billing, containers, testing, and conclusion

Suggested speaking balance: Hassan 14–16 minutes, Tania 8–10 minutes, Faria 10–12 minutes, followed by the integrated demonstration and viva.

B. Recommended Live Demo Sequence

Pre-demo readiness

1. Start the containerized platform and confirm PostgreSQL, migration, backend, frontend, Prometheus, and Grafana health.
2. Use a prepared developer account with a valid GitHub App installation and a small demonstration repository.
3. Confirm required AWS role, region, state backend, GitHub workflow permissions, and billing mode before entering the room.
4. Keep a known successful project and preserved lifecycle evidence available as a fallback.
5. Open browser tabs for DeployGuard, the public deployed application, Grafana, and GitHub workflow evidence.

Live flow

1. **Login and dashboard:** Show authenticated entry, active status, project summary, and attention indicators.
2. **Create project:** Select the GitHub installation, repository, source, and service directory.
3. **Configure service:** Paste non-sensitive demo environment values, confirm the detected port, and optionally attach a managed database.
4. **Review readiness:** Explain service ownership, cost/security admission, and why blocked configuration cannot dispatch.
5. **Start deployment:** Open Pipeline immediately and show the generation and stage transitions.
6. **Inspect evidence:** Show source identity, immutable image identity, Terraform stage, ECS stabilization, and endpoint verification without exposing secrets.
7. **Use the application:** Open the public endpoint and perform one visible application action or health check.
8. **Observe runtime:** Show CloudWatch-derived metrics, a filtered live log stream, Prometheus target health, and the Grafana dashboard.

9. **Troubleshoot:** Open one stored diagnostic and demonstrate evidence-grounded AI advice or its evidence-only fallback.
10. **Show lifecycle control:** Prefer a prepared project to demonstrate release-only redeploy and exact previous-release rollback within the available defense time.
11. **Destroy only if scheduled:** Use explicit confirmation, show project-scoped deletion evidence, and verify that shared platform services remain healthy.

Demo fallback

- If GitHub or AWS is unavailable, use the preserved generation and certification evidence and clearly label it as recorded execution.
- If the AI provider is unavailable, demonstrate the deterministic evidence-only fallback.
- If time is short, do not start a long cloud operation; show the prepared successful lifecycle and explain the live state transitions.

C. Likely Viva Questions

Hassan Sajjad — Cloud, DevOps, Security, Cost, and Reliability

1. **Why does DeployGuard split authority between Terraform, DeployGuard, and ECS?**
   Terraform is best for infrastructure existence and destruction, ECS for low-level rolling runtime behavior, and DeployGuard for product-level admission, verification, promotion, and rollback. The split prevents two systems from fighting over the active release.

2. **How do you guarantee that the deployed code matches the selected source?**
   The source is resolved to an exact revision, the image is published immutably, and the generation links source identity,

image digest, configuration revision, task revision, and verification evidence.

3. **Why is a stable release different from a deployment generation?**
   A generation is an attempt. A stable release is a generation that completed required runtime and public verification. Failed attempts remain visible but never replace the stable pointer.

4. **How are long-lived AWS access keys avoided?**
   GitHub Actions exchanges its workload identity for short-lived AWS credentials through OpenID Connect. Capability admission separately verifies that the role has the operations required for the requested action.

5. **What prevents Terraform from reverting a direct ECS redeployment?**
   The application service lifecycle ignores externally managed desired count and active task revision. Terraform retains resource and destroy ownership while ECS and DeployGuard retain runtime-release ownership.

6. **How does database readiness avoid blocking unrelated services?**
   Readiness is evaluated per attached service. A database-attached app begins at zero tasks and is raised after its own prerequisite succeeds; services without that dependency follow their normal path.

7. **What makes rollback deterministic?**
   It selects an exact prior stable release and task revision with compatible recorded runtime identity. It does not rebuild from source or infer old configuration.

8. **How is cloud cost controlled?**
   The platform produces a structured estimate, associates it with the proposed deployment, evaluates thresholds and approvals, and keeps AWS cost prediction separate from subscription entitlement.

9. **How do you classify deployment failures?**

Each diagnostic records stage, deterministic code, owner, service scope, cause, evidence, and retry guidance. Owners are platform, repository application, external provider, or unverified.

10. **What is the strongest limitation in the current evidence?**
    Some important scenarios still need fresh live-cloud execution, particularly dynamic MySQL grants, MongoDB, database-failure isolation, topology reapply ownership, and repeat-destroy idempotency. They are not presented as completed proof.

Tania Khawar — Frontend, Monitoring, and AI

1. **How does the frontend know a deployment actually succeeded?**
   It renders persisted backend state and cloud-derived evidence. Browser state alone cannot promote a release or declare success.

2. **Why show both current generation and stable release?**
   The distinction lets users see an in-progress or failed attempt without confusing it with the last verified release serving as the safe reference.

3. **How are multi-service settings kept understandable?**
   Configuration is grouped by service directory, with separate environment values, runtime contract, and database attachment. The review step summarizes those boundaries before dispatch.

4. **Why use server-sent events for logs?**
   Logs mainly flow from server to browser, so server-sent events provide a simple streaming channel with built-in reconnection behavior and work well beside ordinary authenticated APIs.

5. **What is the role of CloudWatch compared with Prometheus and Grafana?**
   CloudWatch supplies AWS runtime logs and metrics. Prometheus exposes and collects control-plane metrics, while Grafana provides operator dashboards across those operational signals.

6. **Can the AI assistant change infrastructure or retry a deployment?**

No. It provides advisory analysis. Deterministic services remain responsible for ownership, retry eligibility, lifecycle actions, and release promotion.

7. **How do you reduce hallucination risk?**
   Input evidence is bounded and sanitized, output follows a schema, cited evidence identifiers are validated, authoritative diagnostic fields cannot be overwritten, and invalid responses fall back to deterministic evidence.

8. **How are secrets protected in the troubleshooting interface?**
   Server-side sanitization removes token, password, key, authorization, and secret-URL patterns before evidence is displayed or sent to the model provider.

9. **How does the UI handle long-running actions?**
   Actions create backend operations with explicit state. The UI polls authoritative summaries and streams logs where appropriate, so refreshes and reconnects do not lose the actual operation.

10. **What did frontend testing prove?**
    It covered protected routes, lifecycle projections, cross-page data consistency, monitoring, billing, and admin surfaces. An isolated full-stack run reached READY, while unavailable external GitHub installation was honestly recorded as an expected blocker.

Faria Fatima — Backend, Data, Integration, and Product Operation

1. **Why use PostgreSQL and Redis together?**
   PostgreSQL is the durable system of record. Redis and BullMQ provide efficient job scheduling, retry, backoff, and worker coordination for long operations.

2. **What happens if a worker restarts during deployment?**
   Queue retry policy and persisted operation evidence allow work to resume or reconcile. Durable terminal state is not dependent on one worker process remaining alive.

3. **How is GitHub repository access different from user login?**
   User login establishes identity. Repository access uses a scoped, short-lived GitHub App installation token for repositories granted to that installation.

4. **How does source ingestion avoid checking out the wrong code?**
   It performs a shallow fetch of the selected reference, resolves an exact revision, validates repository paths, and records that identity in the deployment contract.

5. **How are environment values stored?**
   They are service-scoped and encrypted with authenticated encryption. Plaintext values are revealed only to authorized execution paths and are excluded from ordinary responses and evidence.

6. **Why are migrations a separate one-shot container?**
   Schema preparation completes before the backend becomes healthy, making startup order explicit and keeping schema changes reproducible across machines.

7. **How are API inputs secured?**
   Global DTO validation whitelists expected fields and rejects unknown input. This is combined with authenticated sessions, role checks, restricted CORS, HTTP security headers, and domain-level authorization.

8. **How do billing webhooks become trustworthy state?**
   The backend verifies the provider signature before applying events, persists customer and subscription state, and enforces entitlements server-side.

9. **Why retain audit logs if workflow evidence already exists?**
   Workflow evidence explains technical execution; audit logs explain human and system actions: who initiated them, which resource changed, and whether the request succeeded.

10. **What makes the container setup portable?**
    Services, health checks, migration ordering, ports, and persistent volumes are declared together. Machine-specific credentials stay

in external environment configuration rather than inside images.

D. Architecture Facts to Memorize

- DeployGuard is a deterministic deployment control plane, not merely a CI/CD dashboard.
- Frontend: React single-page application built with Vite.
- Backend: NestJS and TypeScript with modular APIs and workers.
- Durable data: PostgreSQL through TypeORM entities and migrations.
- Asynchronous work: Redis and BullMQ.
- Repository access: GitHub App installation tokens with short lifetimes.
- AWS authentication: GitHub Actions OpenID Connect and short-lived credentials.
- Image registry: immutable Amazon ECR repositories and digest-based identity.
- Infrastructure engine: Terraform with encrypted, versioned S3 state and native lockfiles.
- Application runtime: Amazon ECS on Fargate behind Application Load Balancers.
- Logs and cloud metrics: Amazon CloudWatch.
- Platform metrics and dashboards: Prometheus and Grafana.
- Runtime secrets: encrypted control-plane storage and AWS Secrets Manager where platform-managed credentials are required.
- Database persistence: encrypted EFS with encryption in transit.
- Database discovery: private service discovery.
- Supported database contracts: PostgreSQL, MySQL, and MongoDB.
- MySQL has an additional application-user grant-reconciliation requirement.
- Build Target describes how one service is built and started.
- Runtime Config Revision freezes the deploy-time runtime contract.
- Deployment Generation is one attempt; Stable Release is a verified generation.
- Failure evidence includes code, stage, owner, service, cause, and

retry guidance.
- Port resolution follows explicit evidence and fails closed on contradictions.
- Trivy provides image vulnerability evidence and policy enforcement.
- FinOps estimates inform admission; they do not guarantee the final provider bill.
- Stripe integration is demonstrated in test mode.
- AI is advisory, evidence-bounded, schema-validated, and has a deterministic fallback.
- Release-only redeployment updates ECS without a full infrastructure apply.
- Manual rollback selects an exact previous stable task revision without rebuilding.
- Destroy is project-scoped, state-aware, and verified against cloud inventory.
- Structural verification and live-cloud certification are reported separately.

E. Screenshots Needed for the Final Deck

Priority	Screenshot	What Must Be Visible	Preparation Notes
1	Login or landing page	Product identity and clear entry action	Use clean browser chrome and no personal bookmarks
2	Dashboard	Active deployments, attention state, and recent projects	Seed a small set of believable projects

3	GitHub connection	Connected installation or repository selector	Hide account identifiers not needed for defense
4	New Project — repository	Repository, source, and service-directory selection	Crop internal implementation labels
5	New Project — configuration	Per-service environment input, port, and database option	Use non-secret demo values only
6	Deployment readiness	Summary and any security/cost/configuration gates	Prefer a ready example and one clearly explained blocked example
7	Pipeline	Generation, current stage, terminal result, and evidence	Choose a completed deployment with readable stage names
8	Project Overview	Public URL, current state, stable release, and active operation	Ensure the endpoint is healthy before capture
9	Infrastructure	ALB, ECS services, targets, region, database, and supporting resources	Capture the technical-details expansion if readable
10	Monitoring charts	CPU, memory, latency, and target health	Use a time window containing real activity

11	Live logs	Filtered CloudWatch stream and reconnect/status controls	Remove tokens, secret URLs, and noisy irrelevant lines
12	Prometheus	Healthy scrape targets or a representative metric query	Keep bearer credentials out of the screenshot
13	Grafana	Provisioned DeployGuard dashboard with active panels	Set a useful time range and confirm data source health
14	Troubleshooting	Structured diagnostic, evidence references, and AI advice	Include deterministic owner and retry guidance
15	Billing	Test-mode label, plan, usage, and invoice area	Do not show real payment or personal data
16	Admin readiness	Platform services and readiness status	Use an administrator session only for this capture
17	Audit log	Actor, action, resource, status, and sanitized metadata	Confirm confidential values are absent
18	GitHub workflow	Major delivery stages and successful result	Hide repository details if the deck will be public
19	AWS ECS service	Running tasks, deployment state, and task revision	Match it to the project shown in DeployGuard

20	AWS load balancer	Healthy target and public endpoint	Capture the matching service and region
21	ECR image	Immutable image and digest identity	Do not expose unrelated repositories
22	Terraform evidence	Successful plan/ apply summary without secrets	Prefer DeployGuard evidence over raw terminal output
23	Release-only redeploy	New task revision and unchanged infrastructure path	Pair with the prior stable-release screenshot
24	Rollback	Selected historical release and successful verification	Show that no source rebuild occurred
25	Destroy verification	Project resources absent and shared platform healthy	Capture before/ after identifiers without hashes
26	Container stack health	All local product services healthy	Include frontend, backend, database, monitoring, and migration completion

Before final export, crop all secrets, tokens, personal email addresses, repository identifiers not intended for disclosure, cloud account numbers, exact source hashes, and development branch labels. Use consistent browser dimensions and timestamp the evidence privately even if the timestamp is not shown on the slide.